# Null Object

Null Object · Safe logger when monitoring disabled · Build with Waqar

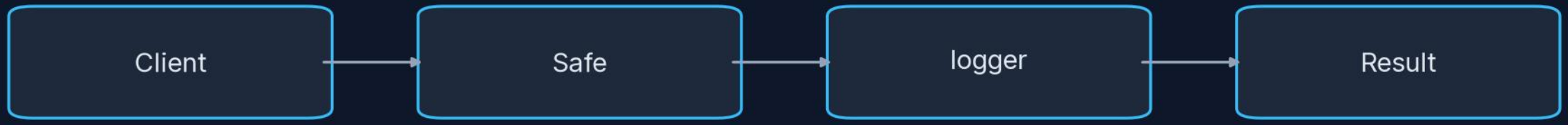
# Today's Hook

- Problem: Safe logger when monitoring disabled
- Pattern: Null Object — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Safe logger when monitoring disabled
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




Null Object: Safe logger when monitoring disabled

# Real-World Scenario

- Running example: Safe logger when monitoring disabled
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.nullobject.problem;

/** Naive: logger may be null — every call site null-checks. */
public final class NaiveOrderService {
  private final Logger log;
  public NaiveOrderService(Logger log){ this.log=log; }
  public void place(String id){ if(log!=null) log.info("placed "+id); }
}
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```
package com.buildwithwaqar.patterns.enterprise.nullobject.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

interface Logger { void info(String m); }
final class ConsoleLogger implements Logger { private final List<String> lines=new ArrayList<>();
  public void info(String m){ lines.add(m); } List<String> lines(){ return List.copyOf(lines); } }
final class NullLogger implements Logger { public void info(String m){ /* no-op */ } }
final class OrderService {
  private final Logger log; OrderService(Logger log){ this.log=Objects.requireNonNull(log); }
  void place(String id){ log.info("placed "+id); }
}
```

# Runnable main

```
package com.buildwithwaqar.patterns.enterprise.nullobject.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Null Object: safe logger ===");
    System.out.println("--- naive null check ---");
new NaiveOrderService(null).place("o1");
System.out.println("--- null object ---");
var console=new ConsoleLogger();
new OrderService(console).place("o1");
new OrderService(new NullLogger()).place("o2");
System.out.println(console.lines());
  }
}
```

# Walkthrough Output

```
=== Null Object: safe logger ===
--- naive null check ---
--- null object ---
[placed o1]
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Null Object.

# Related Patterns

- Contrast Null Object with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → Null Object → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.